AIMLC ZC416 - Mathematical Foundations for Machine Learning

Assignment 1: Linear Systems and Matrix Decompositions

Student Details:

- **Name:** Ashwani Singh
- **BITS ID:** 2025AC05272

Overview of Guidelines

As per the course guidelines and general queries clarifications:

1. **No Built-in Matrix Functions:** All operations for REF, RREF, LU, Cholesky, and QR are implemented completely from scratch using basic Python loops, conditioning, and element-level arithmetic. Libraries like **NumPy** are only used for basic data storage, array indexing, shape inspection, and simple scalar arithmetic.
2. **Aesthetics & Premium Format:** This notebook uses beautiful LaTeX equations, well-formatted console printouts, and clear highlighted instructions for the handwritten submission.
3. **Handwritten Submissions:** Sections marked with ✍️ **[HANDWRITTEN SUBMISSION NOTE]** provide structured guidelines and results that you should write down in your final handwritten PDF submission (`BITSID.pdf`).

Core Custom Linear Algebra Utilities

To ensure absolute compliance with the "no built-in functions" instruction, we first define our own utility functions for fundamental operations such as matrix multiplication, transpose, vector norms, and beautiful matrix printing.

In [1]:
```python
import numpy as np

def custom_mat_mul(A, x):
    """
    Computes matrix-vector multiplication Ax.
    A: 2D numpy array of shape (m, n)
    x: 1D or 2D numpy array of shape (n,) or (n, 1)
    """
    m, n = A.shape
    # Flatten x to 1D if needed for easier indexing
    x_flat = x.flatten()
    res = np.zeros(m)
    for i in range(m):
```

```
        for j in range(n):
            res[i] += A[i, j] * x_flat[j]
    return res if len(x.shape) == 1 else res.reshape(-1, 1)

def custom_mat_mul_mat(A, B):
    """
    Computes matrix-matrix multiplication AB.
    A: shape (m, n)
    B: shape (n, p)
    """
    m, n = A.shape
    n2, p = B.shape
    assert n == n2, "Inner dimensions must match for matrix multiplicatio
    res = np.zeros((m, p))
    for i in range(m):
        for j in range(p):
            for k in range(n):
                res[i, j] += A[i, k] * B[k, j]
    return res

def custom_mat_transpose(A):
    """
    Computes the transpose of a matrix A.
    """
    m, n = A.shape
    res = np.zeros((n, m))
    for i in range(m):
        for j in range(n):
            res[j, i] = A[i, j]
    return res

def custom_vector_norm(x):
    """
    Computes the L2 norm of a vector x.
    """
    return np.sqrt(np.sum(x ** 2))

def custom_matrix_norm_frobenius(A):
    """
    Computes the Frobenius norm of a matrix A.
    """
    return np.sqrt(np.sum(A ** 2))

def print_matrix(name, M, decimals=4):
    """
    Prints a matrix M beautifully with rounded float values for readabili
    Cleans up extremely small values near zero.
    """
    print(f"--- {name} ---")
    m, n = M.shape
    for i in range(m):
        row_str = "[ "
        for j in range(n):
            val = M[i, j]
            # Clean up tiny floating point values
            if abs(val) < 1e-12:
                val = 0.0
            row_str += f"{val: .4f}  "
        row_str += "]"
```

```
        print(row_str)
    print()
```

Q1) Finding Solutions of Linear Systems

Mathematical Background & Theory

We are given an underdetermined system of linear equations $Ax = b$, where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^{m \times 1}$ with $m < n$. Since the number of variables (n) exceeds the number of equations (m), the system will either have **infinitely many solutions** (if it is consistent) or **no solution** (if it is inconsistent).

To solve this system, we construct the **augmented matrix** $[A \mid b]$ of size $m \times (n + 1)$. We then apply elementary row operations to reduce $[A \mid b]$ to:

1. **Row Echelon Form (REF):**
   - All non-zero rows are above any rows of all zeros.
   - The leading coefficient (pivot) of a non-zero row is strictly to the right of the leading coefficient of the row above it.
   - All entries in a column below a leading coefficient are zero.
2. **Reduced Row Echelon Form (RREF):**
   - It is in REF.
   - The leading entry in each non-zero row is 1 (normalized pivot).
   - Each leading 1 is the only non-zero entry in its column.

Numerical Stability: Partial Pivoting

To avoid division by zero and improve numerical stability, we implement **partial pivoting**. Before eliminating entries below the active element in column j, we scan the rows from the current row i down to $m - 1$ to find the row with the largest absolute value in column j. We swap this row with the active row i before proceeding. If the largest entry in the column is zero (numerically $< 10^{-9}$), it indicates a **free variable** column, and we move to the next column without incrementing our active row.

Q1(a) Custom REF and RREF Code Implementation

In [2]:
```
def compute_ref(A, b):
    """
    Constructs the augmented matrix [A | b] and reduces it to Row Echelon
    using Gaussian elimination with partial pivoting.
    Returns the REF augmented matrix and a list of pivot tuples (row, col
    """
    m, n = A.shape
    # Construct the augmented matrix [A | b]
    M = np.hstack((A.astype(np.float64), b.astype(np.float64).reshape(-1,

    pivot_row = 0
    pivots = []
```

```
    for col in range(n):
        if pivot_row >= m:
            break

        # Find the row with maximum absolute entry in active column 'col'
        max_row = pivot_row
        for r in range(pivot_row + 1, m):
            if abs(M[r, col]) > abs(M[max_row, col]):
                max_row = r

        # If pivot value is virtually zero, this is a non-pivot column
        if abs(M[max_row, col]) < 1e-9:
            continue

        # Swap rows to place the largest pivot in the current pivot row
        if max_row != pivot_row:
            # Swap row max_row and pivot_row
            temp = M[pivot_row].copy()
            M[pivot_row] = M[max_row]
            M[max_row] = temp

        pivots.append((pivot_row, col))

        # Eliminate entries below the pivot
        for r in range(pivot_row + 1, m):
            factor = M[r, col] / M[pivot_row, col]
            M[r] = M[r] - factor * M[pivot_row]

        pivot_row += 1

    return M, pivots

def compute_rref(A, b):
    """
    Reduces the augmented matrix [A | b] to Reduced Row Echelon Form (RRE
    Returns the RREF augmented matrix and a list of pivot tuples (row, co
    """
    # First compute the Row Echelon Form (REF)
    M, pivots = compute_ref(A, b)
    m, n = A.shape

    # Back-substitution: start from the last pivot and eliminate upwards
    for r, c in reversed(pivots):
        # Normalize the pivot row so that the pivot entry becomes exactly
        pivot_val = M[r, c]
        M[r] = M[r] / pivot_val

        # Eliminate entries in column 'c' in all rows above the pivot row
        for row in range(r):
            factor = M[row, c]
            M[row] = M[row] - factor * M[r]

    return M, pivots
```

Q1(b) Identifying Pivot Columns, Particular, and Homogeneous Solutions

Once the augmented matrix $[A \mid b]$ is reduced to RREF $[R \mid d]$, we can fully characterize the solution space.

1. Identifying Pivot and Non-Pivot Columns

- **Pivot Columns:** Columns $j \in \{0, \dots, n-1\}$ that contain a leading 1 in RREF. These correspond to the **pivot variables** (dependent variables).
- **Non-Pivot Columns:** All remaining columns in A. These correspond to **free variables** (independent variables), which can take any arbitrary scalar value.

2. Checking Consistency

The system is consistent if and only if there is **no row** in RREF $[R \mid d]$ that has all zeros in the coefficient part but a non-zero value in the augmented column d. Mathematically, if there is a row index r such that $R[r, j] = 0$ for all $j = 0, \dots, n-1$ but $d[r] \neq 0$, the system is **inconsistent** (equivalent to $0 = 1$), and no solution exists.

3. Particular Solution (x_p)

To find a particular solution $x_p \in \mathbb{R}^n$, we exploit the free variables. We set **all free variables to** 0. Then, for each pivot variable x_c associated with the pivot (r, c) in our pivots list, the equation in row r simplifies directly to:

$$x_c = d[r]$$

Thus, x_p is a vector where:

- $x_p[c] = d[r]$ for each pivot (r, c).
- $x_p[j] = 0$ for all free column indices j.

4. Homogeneous Solutions ($Ax = 0$)

The solutions to $Ax = 0$ represent the null space (kernel) of A. Since there are k pivots, there are $d = n - k$ free variables. To construct a basis $\{v_1, \dots, v_d\}$ for the homogeneous solution space: For each free variable column $f \in \text{free\_cols}$, we construct a basis vector v_f by:

1. Setting $x_f = 1$.
2. Setting all other free variables $j \in \text{free\_cols} \setminus \{f\}$ to 0.
3. Solving for the pivot variables in RREF using $Rx = 0$: For each pivot (r, c), we have:

$$x_c + \sum_{j \in \text{free\_cols}} R[r, j] x_j = 0 \implies x_c = -R[r, f]$$

Hence, each null space basis vector $v_f \in \mathbb{R}^n$ is constructed as:

- $v_f[f] = 1.0$.
- $v_f[j] = 0.0$ for all other free columns $j \neq f$.

- $v_f[c] = -R[r, f]$ for each pivot (r, c).

Q1(b) Solution Space Analysis Implementation

In [3]:

```python
def analyze_solution_space(A, b):
    """
    Solves Ax = b by computing RREF and extracting pivot/free columns,
    particular solution, and homogeneous null space basis.
    """
    m, n = A.shape
    M, pivots = compute_rref(A, b)

    # Separate R (coefficient part) and d (augmented column)
    R = M[:, :n]
    d = M[:, n]

    # Check for consistency
    is_consistent = True
    for r in range(m):
        all_zeros = True
        for c in range(n):
            if abs(R[r, c]) > 1e-9:
                all_zeros = False
                break
        if all_zeros and abs(d[r]) > 1e-9:
            is_consistent = False
            break

    if not is_consistent:
        return {
            "consistent": False,
            "RREF": M,
            "pivots": pivots,
            "pivot_cols": [c for r, c in pivots],
            "free_cols": [c for c in range(n) if c not in [p[1] for p in
            "particular_sol": None,
            "homogeneous_basis": []
        }

    # Identify pivot columns and free columns
    pivot_cols = [c for r, c in pivots]
    free_cols = [c for c in range(n) if c not in pivot_cols]

    # 1. Compute particular solution xp
    x_p = np.zeros(n)
    for r, c in pivots:
        x_p[c] = d[r]

    # 2. Compute basis vectors for homogeneous solution Ax = 0
    homogeneous_basis = []
    for f in free_cols:
        v = np.zeros(n)
        v[f] = 1.0
        for r, c in pivots:
            # In Ax = 0, the pivot variable x_c = - R[r, f] * x_f
            v[c] = -R[r, f]
        homogeneous_basis.append(v)
```

```
    return {
        "consistent": True,
        "RREF": M,
        "pivots": pivots,
        "pivot_cols": pivot_cols,
        "free_cols": free_cols,
        "particular_sol": x_p,
        "homogeneous_basis": homogeneous_basis
    }
```

Q1(c) Randomized System Generation, Solution, and Verification

Now we construct a random 6×9 matrix A and a suitable vector b. To ensure that the system is **guaranteed to be consistent**, we:

1. Generate $A \in \mathbb{R}^{6\times 9}$ using random integers in $[-5, 5]$.
2. Generate a random true solution $x_{\text{true}} \in \mathbb{R}^{9\times 1}$ of integers in $[-3, 3]$.
3. Compute the consistent vector $b = Ax_{\text{true}}$ using our custom matrix multiplication.

We then solve this system and verify the **general solution**:

$$x_{\text{general}} = x_p + \sum_{i=1}^{d} c_i v_i$$

by picking random constants c_i and verifying that $Ax_{\text{general}} \approx b$ and $Av_i \approx 0$ for all null space vectors.

> ### ✍️ [HANDWRITTEN SUBMISSION NOTE]
>
> **For your handwritten PDF assignment submission (`BITSID.pdf`), you should write:**
>
> 1. **Random Matrix A & Vector b:** Copy the printed random matrix A and vector b from the cell output below.
> 2. **REF and RREF Matrices:** Write down the computed Augmented Row Echelon Form (REF) and Reduced Row Echelon Form (RREF) matrices.
> 3. **Pivot Analysis:** List the identified Pivot Columns (associated with leading 1s) and Free Columns (associated with free variables).
> 4. **Solution Formulation:** Write down the particular solution x_p and the basis vectors $\{v_1, v_2, v_3\}$ for the homogeneous system $Ax = 0$.
> 5. **General Solution:** Express the general solution as $x = x_p + c_1v_1 + c_2v_2 + c_3v_3$.
> 6. **Verification Statement:** State that the custom code verified the general solution by showing that $Ax_p \approx b$ and $Av_i \approx 0$, reporting the numerical residual errors printed below.

Q1(c) Executing Randomized 6×9 Linear System Solution

In [4]:

```python
# Seed for reproducibility of random generation in this run
np.random.seed(42)

# 1. Generate a random 6 x 9 matrix A with integer entries in [-5, 5]
A_rand = np.random.randint(-5, 6, size=(6, 9))

# 2. Generate a random 9-dimensional true solution vector with integers i
x_true = np.random.randint(-3, 4, size=9)

# 3. Compute consistent vector b = A * x_true using custom matrix-vector
b_rand = custom_mat_mul(A_rand, x_true)

# 4. Print inputs
print_matrix("Random Matrix A (6 x 9)", A_rand)
print("Random consistent target vector b:")
print(b_rand)
print()

# 5. Perform solution analysis
analysis = analyze_solution_space(A_rand, b_rand)

# 6. Print REF & RREF
ref_M, _ = compute_ref(A_rand, b_rand)
print_matrix("Augmented Matrix [A | b] in Row Echelon Form (REF)", ref_M)
print_matrix("Augmented Matrix [A | b] in Reduced Row Echelon Form (RREF)

# 7. Print Pivot & Free Columns
print(f"Pivot Columns (0-indexed): {analysis['pivot_cols']}")
print(f"Free Columns (0-indexed): {analysis['free_cols']}")
print()

# 8. Print Particular Solution
x_p = analysis["particular_sol"]
print("Particular Solution x_p:")
print(x_p)
print()

# 9. Print Homogeneous Solutions (Null Space Basis)
basis = analysis["homogeneous_basis"]
print(f"Homogeneous Solution Space Dimension (Nullity) = {len(basis)}")
for idx, v in enumerate(basis):
    print(f"Null Space Basis Vector v_{idx+1}:")
    print(v)
print()

# 10. Construct General Solution & Verify
print("--- GENERAL SOLUTION VERIFICATION ---")
# Pick arbitrary constants c_i
c_coeffs = [2.5, -1.2, 0.8]  # Rank = 6, Free variables = 3
print(f"Choosing arbitrary free variables coefficients: c = {c_coeffs}")

x_gen = x_p.copy()
for i, coeff in enumerate(c_coeffs):
    x_gen += coeff * basis[i]

print("Constructed General Solution x_general:")
```

```
print(x_gen)
print()

# Verify A * x_general = b
A_x_gen = custom_mat_mul(A_rand, x_gen)
residual_xp = custom_vector_norm(custom_mat_mul(A_rand, x_p) - b_rand)
residual_gen = custom_vector_norm(A_x_gen - b_rand)
print(f"Particular solution residual ||A*x_p - b||_2 = {residual_xp:.4e}"
print(f"General solution residual ||A*x_gen - b||_2 = {residual_gen:.4e}"

# Verify A * v_i = 0 for each basis vector
for i, v in enumerate(basis):
    residual_vi = custom_vector_norm(custom_mat_mul(A_rand, v))
    print(f"Homogeneous basis vector v_{i+1} residual ||A*v_{i+1}||_2 = {
```

```
--- Random Matrix A (6 x 9) ---
[  1.0000  -2.0000   5.0000   2.0000  -1.0000   1.0000   4.0000  -3.0000
1.0000  ]
[  5.0000   5.0000   2.0000  -1.0000  -2.0000   2.0000   2.0000  -3.0000
0.0000  ]
[ -1.0000  -4.0000   2.0000   0.0000  -4.0000  -1.0000  -5.0000   4.0000
0.0000  ]
[  3.0000  -5.0000   5.0000   5.0000   4.0000  -3.0000   1.0000  -2.0000
3.0000  ]
[ -3.0000  -1.0000  -3.0000   1.0000  -1.0000   3.0000   1.0000  -4.0000
-2.0000  ]
[  3.0000  -4.0000   4.0000   3.0000   4.0000  -1.0000  -4.0000  -2.0000
1.0000  ]

Random consistent target vector b:
[ 12.   8. -11.  26. -14.  20.]

--- Augmented Matrix [A | b] in Row Echelon Form (REF) ---
[  5.0000   5.0000   2.0000  -1.0000  -2.0000   2.0000   2.0000  -3.0000
0.0000   8.0000  ]
[  0.0000  -8.0000   3.8000   5.6000   5.2000  -4.2000  -0.2000  -0.2000
3.0000   21.2000  ]
[  0.0000   0.0000   3.1750   0.1000  -2.5500   2.1750   3.6750  -2.3250
-0.1250   2.4500  ]
[  0.0000   0.0000   0.0000  -2.3307  -5.5669   0.3071  -5.6535   4.1890
-1.0866  -18.1024  ]
[  0.0000   0.0000   0.0000   0.0000  -5.9459   3.9730  -1.2973  -3.1892
-2.1351  -17.4324  ]
[  0.0000   0.0000   0.0000   0.0000   0.0000   3.8665  -2.0227  -4.4830
-2.2301  -2.6335  ]

--- Augmented Matrix [A | b] in Reduced Row Echelon Form (RREF) ---
[  1.0000   0.0000   0.0000   0.0000   0.0000   0.0000  -2.1533   0.0553
-0.1974  -0.1105  ]
[  0.0000   1.0000   0.0000   0.0000   0.0000   0.0000   2.7136  -0.5286
0.3800   2.0572  ]
[  0.0000   0.0000   1.0000   0.0000   0.0000   0.0000   1.3262  -0.0860
0.3204   3.1719  ]
[  0.0000   0.0000   0.0000   1.0000   0.0000   0.0000   2.6705  -1.3807
0.4530   1.7615  ]
[  0.0000   0.0000   0.0000   0.0000   1.0000   0.0000  -0.1314  -0.2384
-0.0263   2.4767  ]
[  0.0000   0.0000   0.0000   0.0000   0.0000   1.0000  -0.5231  -1.1594
-0.5768  -0.6811  ]

Pivot Columns (0-indexed): [0, 1, 2, 3, 4, 5]
Free Columns (0-indexed): [6, 7, 8]

Particular Solution x_p:
[-0.11050698  2.05716385  3.1719324   1.7614989   2.4767083  -0.68111683
  0.          0.          0.        ]

Homogeneous Solution Space Dimension (Nullity) = 3
Null Space Basis Vector v_1:
[ 2.15326965 -2.71359295 -1.32623071 -2.67053637  0.13137399  0.52314475
  1.          0.          0.        ]
Null Space Basis Vector v_2:
[-0.05525349  0.52858193  0.0859662   1.38074945  0.23835415  1.15944159
  0.          1.          0.        ]
Null Space Basis Vector v_3:
```

```
[ 0.19735489 -0.3800147  -0.32035268 -0.45304923  0.02630419  0.57678178
  0.          0.          1.        ]

--- GENERAL SOLUTION VERIFICATION ---
Choosing arbitrary free variables coefficients: c = [2.5, -1.2, 0.8]
Constructed General Solution x_general:
[ 5.49685525 -5.66512858 -0.50308597 -6.93418075  2.54016165 -0.30315944
  2.5        -1.2         0.8       ]

Particular solution residual ||A*x_p - b||_2 = 6.6465e-15
General solution residual ||A*x_gen - b||_2 = 1.4431e-14
Homogeneous basis vector v_1 residual ||A*v_1||_2 = 2.6668e-15
Homogeneous basis vector v_2 residual ||A*v_2||_2 = 2.0351e-15
Homogeneous basis vector v_3 residual ||A*v_3||_2 = 9.4857e-16
```

Q2) Matrix Decompositions

Matrix decompositions factorize a matrix into a product of simpler matrices, which are easier to work with, invert, or analyze. In this section, we implement three major decompositions entirely from scratch:

1. **LU Decomposition:** Factorizes $A = LU$ using elementary row operations and their corresponding elementary matrices.
2. **Cholesky Decomposition:** Factorizes a symmetric positive definite matrix $A = LL^T$.
3. **QR Decomposition:** Factorizes an $m \times n$ matrix A ($m > n$) with linearly independent columns into $A = QR$ using the Gram-Schmidt process.

Q2(a) LU Decomposition using Elementary Matrices

Theory & Mathematical Derivation

LU decomposition factorizes a square matrix $A \in \mathbb{R}^{n \times n}$ into a lower triangular matrix L (with 1s on the diagonal) and an upper triangular matrix U.

For a **symmetric and positive definite** matrix A, Gaussian elimination can always be completed **without row swaps** (no pivoting required). The row elimination process of converting A to an upper triangular matrix U can be written as a sequence of elementary row operations. Specifically, to eliminate the entry in row i, column j (where $i > j$), we perform the operation:

$$R_i \leftarrow R_i - l_{ij} R_j \quad \text{where } l_{ij} = \frac{A[i,j]}{A[j,j]}$$

This row operation is mathematically equivalent to multiplying A on the left by an **elementary matrix** E_{ij}. The elementary matrix E_{ij} is formed by taking the identity matrix I_n and replacing the entry at row i, column j with $-l_{ij}$:

$$E_{ij} = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{ij} & \ddots & \\ & & & & 1 \end{bmatrix}$$

If there are k elimination steps, the final upper triangular matrix U is:

$$E_k E_{k-1} \dots E_1 A = U$$

To reconstruct A, we multiply both sides by the inverses of the elementary matrices in reverse order:

$$A = (E_1^{-1} E_2^{-1} \dots E_k^{-1}) U = LU$$

Where each **inverse elementary matrix** E_{ij}^{-1} is simply I_n with the sign of the multiplier reversed ($+l_{ij}$ instead of $-l_{ij}$):

$$E_{ij}^{-1} = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & l_{ij} & \ddots & \\ & & & & 1 \end{bmatrix}$$

The lower triangular matrix L is computed as the sequential product of these inverse elementary matrices:

$$L = E_1^{-1} E_2^{-1} \dots E_k^{-1}$$

Since we are performing the operations in a nested column-by-column loop, this product L is exactly lower triangular with the multipliers l_{ij} at row i, column j and 1s on the diagonal.

> ## ✍️ [HANDWRITTEN SUBMISSION NOTE]
>
> **For your handwritten PDF assignment submission (`BITSID.pdf`), you should write:**
>
> 1. **Matrix A:** Write down the symmetric positive definite matrix A generated below.
> 2. **Elementary Matrices:** Show how the elementary matrices E_{ij} are constructed for each row operation (with 1s on the diagonal and the negative multiplier at row i, column j). Provide a sample E_{ij} and E_{ij}^{-1}.
> 3. **Sequence of Operations:** Write down the sequence of operations that yields L and U.

4. **Decomposition Result:** Write down the final lower triangular matrix L and upper triangular matrix U.
5. **Verification Proof:** Show that multiplying L and U reconstructs A exactly ($LU = A$).

Q2(a) Custom LU Decomposition with Elementary Matrices Implementation

In [5]:
```python
def lu_decomposition_elementary(A_orig):
    """
    Computes the LU decomposition of a square matrix A using elementary m
    Returns L, U, list of elementary matrices, and list of inverse elemen
    """
    n = A_orig.shape[0]
    # Initialize U as a copy of A, and L as the identity matrix
    U = A_orig.copy().astype(np.float64)
    L = np.eye(n)

    elementary_matrices = []
    inverse_elementary_matrices = []

    # Run Gaussian elimination column by column
    for j in range(n):
        for i in range(j + 1, n):
            if abs(U[j, j]) < 1e-9:
                raise ValueError("Zero pivot encountered. LU decompositio

            # Calculate multiplier l_ij
            multiplier = U[i, j] / U[j, j]

            # Construct the elementary matrix E_ij
            E_ij = np.eye(n)
            E_ij[i, j] = -multiplier
            elementary_matrices.append((i, j, E_ij))

            # Construct the inverse elementary matrix E_ij_inv
            E_ij_inv = np.eye(n)
            E_ij_inv[i, j] = multiplier
            inverse_elementary_matrices.append((i, j, E_ij_inv))

            # Apply row operation to U: U = E_ij * U using custom matrix-
            U = custom_mat_mul_mat(E_ij, U)

            # Accumulate inverse elementary matrix into L: L = L * E_ij_i
            L = custom_mat_mul_mat(L, E_ij_inv)

    return L, U, elementary_matrices, inverse_elementary_matrices

# --- Test and Verification of Q2(a) ---
np.random.seed(15)

# 1. Construct a symmetric and positive definite matrix A of size 4 x 4
# B is a random matrix, A = B * B^T is guaranteed symmetric positive semi
# We add 4*I to ensure it is strictly positive definite and has clean int
B = np.random.randint(1, 4, size=(4, 4))
A_sym = custom_mat_mul_mat(B, custom_mat_transpose(B)) + 4 * np.eye(4)
```

```python
print_matrix("Symmetric & Positive Definite Matrix A (4 x 4)", A_sym)

# 2. Perform LU Decomposition with elementary matrices
L, U, E_list, E_inv_list = lu_decomposition_elementary(A_sym)

# 3. Print the generated elementary matrices
print("=== GENERATION OF ELEMENTARY MATRICES ===")
for idx, (i, j, E_ij) in enumerate(E_list):
    print(f"Step {idx+1}: Eliminate element at ({i+1},{j+1}) using pivot
    print(f"Row Operation: R_{i+1} <- R_{i+1} - ({A_sym[i,j]/A_sym[j,j]:.
    print_matrix(f"Elementary Matrix E_{i+1}{j+1}", E_ij)
    print_matrix(f"Inverse Elementary Matrix E_{i+1}{j+1}^(-1)", E_inv_li
    print("-" * 40)
print()

# 4. Print final lower L and upper U matrices
print_matrix("Lower Triangular Matrix L", L)
print_matrix("Upper Triangular Matrix U", U)

# 5. Verification of A = LU
LU_product = custom_mat_mul_mat(L, U)
print_matrix("Reconstructed Matrix L * U", LU_product)
error_lu = custom_matrix_norm_frobenius(A_sym - LU_product)
print(f"Reconstruction Error ||A - L*U||_F = {error_lu:.4e}")
```

```
--- Symmetric & Positive Definite Matrix A (4 x 4) ---
[  14.0000   11.0000   10.0000   14.0000  ]
[  11.0000   19.0000   11.0000   14.0000  ]
[  10.0000   11.0000   17.0000   16.0000  ]
[  14.0000   14.0000   16.0000   27.0000  ]

=== GENERATION OF ELEMENTARY MATRICES ===
Step 1: Eliminate element at (2,1) using pivot at (1,1)
Row Operation: R_2 <- R_2 - (0.7857) * R_1
--- Elementary Matrix E_21 ---
[  1.0000   0.0000   0.0000   0.0000  ]
[ -0.7857   1.0000   0.0000   0.0000  ]
[  0.0000   0.0000   1.0000   0.0000  ]
[  0.0000   0.0000   0.0000   1.0000  ]

--- Inverse Elementary Matrix E_21^(-1) ---
[  1.0000   0.0000   0.0000   0.0000  ]
[  0.7857   1.0000   0.0000   0.0000  ]
[  0.0000   0.0000   1.0000   0.0000  ]
[  0.0000   0.0000   0.0000   1.0000  ]

----------------------------------------
Step 2: Eliminate element at (3,1) using pivot at (1,1)
Row Operation: R_3 <- R_3 - (0.7143) * R_1
--- Elementary Matrix E_31 ---
[  1.0000   0.0000   0.0000   0.0000  ]
[  0.0000   1.0000   0.0000   0.0000  ]
[ -0.7143   0.0000   1.0000   0.0000  ]
[  0.0000   0.0000   0.0000   1.0000  ]

--- Inverse Elementary Matrix E_31^(-1) ---
[  1.0000   0.0000   0.0000   0.0000  ]
[  0.0000   1.0000   0.0000   0.0000  ]
[  0.7143   0.0000   1.0000   0.0000  ]
[  0.0000   0.0000   0.0000   1.0000  ]

----------------------------------------
Step 3: Eliminate element at (4,1) using pivot at (1,1)
Row Operation: R_4 <- R_4 - (1.0000) * R_1
--- Elementary Matrix E_41 ---
[  1.0000   0.0000   0.0000   0.0000  ]
[  0.0000   1.0000   0.0000   0.0000  ]
[  0.0000   0.0000   1.0000   0.0000  ]
[ -1.0000   0.0000   0.0000   1.0000  ]

--- Inverse Elementary Matrix E_41^(-1) ---
[  1.0000   0.0000   0.0000   0.0000  ]
[  0.0000   1.0000   0.0000   0.0000  ]
[  0.0000   0.0000   1.0000   0.0000  ]
[  1.0000   0.0000   0.0000   1.0000  ]

----------------------------------------
Step 4: Eliminate element at (3,2) using pivot at (2,2)
Row Operation: R_3 <- R_3 - (0.5789) * R_2
--- Elementary Matrix E_32 ---
[  1.0000   0.0000   0.0000   0.0000  ]
[  0.0000   1.0000   0.0000   0.0000  ]
[  0.0000  -0.3034   1.0000   0.0000  ]
[  0.0000   0.0000   0.0000   1.0000  ]
```

```
--- Inverse Elementary Matrix E_32^(-1) ---
[  1.0000   0.0000   0.0000   0.0000  ]
[  0.0000   1.0000   0.0000   0.0000  ]
[  0.0000   0.3034   1.0000   0.0000  ]
[  0.0000   0.0000   0.0000   1.0000  ]

----------------------------------------
Step 5: Eliminate element at (4,2) using pivot at (2,2)
Row Operation: R_4 <- R_4 - (0.7368) * R_2
--- Elementary Matrix E_42 ---
[  1.0000   0.0000   0.0000   0.0000  ]
[  0.0000   1.0000   0.0000   0.0000  ]
[  0.0000   0.0000   1.0000   0.0000  ]
[  0.0000  -0.2897   0.0000   1.0000  ]

--- Inverse Elementary Matrix E_42^(-1) ---
[  1.0000   0.0000   0.0000   0.0000  ]
[  0.0000   1.0000   0.0000   0.0000  ]
[  0.0000   0.0000   1.0000   0.0000  ]
[  0.0000   0.2897   0.0000   1.0000  ]

----------------------------------------
Step 6: Eliminate element at (4,3) using pivot at (3,3)
Row Operation: R_4 <- R_4 - (0.9412) * R_3
--- Elementary Matrix E_43 ---
[  1.0000   0.0000   0.0000   0.0000  ]
[  0.0000   1.0000   0.0000   0.0000  ]
[  0.0000   0.0000   1.0000   0.0000  ]
[  0.0000   0.0000  -0.5716   1.0000  ]

--- Inverse Elementary Matrix E_43^(-1) ---
[  1.0000   0.0000   0.0000   0.0000  ]
[  0.0000   1.0000   0.0000   0.0000  ]
[  0.0000   0.0000   1.0000   0.0000  ]
[  0.0000   0.0000   0.5716   1.0000  ]

----------------------------------------

--- Lower Triangular Matrix L ---
[  1.0000   0.0000   0.0000   0.0000  ]
[  0.7857   1.0000   0.0000   0.0000  ]
[  0.7143   0.3034   1.0000   0.0000  ]
[  1.0000   0.2897   0.5716   1.0000  ]

--- Upper Triangular Matrix U ---
[  14.0000   11.0000   10.0000   14.0000  ]
[  0.0000   10.3571   3.1429   3.0000  ]
[  0.0000   0.0000   8.9034   5.0897  ]
[  0.0000   0.0000   0.0000   9.2215  ]

--- Reconstructed Matrix L * U ---
[  14.0000   11.0000   10.0000   14.0000  ]
[  11.0000   19.0000   11.0000   14.0000  ]
[  10.0000   11.0000   17.0000   16.0000  ]
[  14.0000   14.0000   16.0000   27.0000  ]

Reconstruction Error ||A - L*U||_F = 0.0000e+00
```

Q2(b) Cholesky's Decomposition

Theory & Algorithm

Cholesky decomposition factorizes a symmetric and positive definite matrix $A \in \mathbb{R}^{n \times n}$ into:

$$A = L_{\text{chol}} L_{\text{chol}}^T$$

where L_{chol} is a lower triangular matrix with **strictly positive diagonal elements**.

We implement the **Cholesky-Banachiewicz algorithm**, which solves for the elements of L_{chol} row-by-row using the following formulas:

1. For diagonal elements ($i = j$):

$$L_{ii} = \sqrt{A_{ii} - \sum_{k=0}^{i-1} L_{ik}^2}$$

2. For off-diagonal elements ($i > j$):

$$L_{ij} = \frac{1}{L_{jj}} \left(A_{ij} - \sum_{k=0}^{j-1} L_{ik} L_{jk} \right)$$

Since A is symmetric and positive definite, the value inside the square root for L_{ii} is guaranteed to be strictly positive (> 0), ensuring a real and unique lower triangular matrix.

> ✍️ [HANDWRITTEN SUBMISSION NOTE]
>
> **For your handwritten PDF assignment submission (`BITSID.pdf`), you should write:**
>
> 1. **Cholesky Formula:** State the equations for L_{ii} and L_{ij}.
> 2. **Cholesky Factor:** Write down the computed lower triangular Cholesky factor L_{chol} for the matrix A from Q2(a).
> 3. **Verification:** Show that $L_{\text{chol}} L_{\text{chol}}^T = A$ by multiplying the matrices.

Q2(b) Custom Cholesky Decomposition Implementation

In [6]:
```python
def cholesky_decomposition(A):
    """
    Computes the Cholesky decomposition of a symmetric positive definite
    Returns L_chol such that A = L_chol * L_chol^T.
    """
    n = A.shape[0]
    L_chol = np.zeros((n, n))

    for i in range(n):
        for j in range(i + 1):
            # Sum up terms L[i,k]*L[j,k]
            sum_k = 0.0
```

```
            for k in range(j):
                sum_k += L_chol[i, k] * L_chol[j, k]

            if i == j:
                val = A[i, i] - sum_k
                if val < 0:
                    raise ValueError("Matrix is not positive definite!")
                L_chol[i, j] = np.sqrt(val)
            else:
                L_chol[i, j] = (A[i, j] - sum_k) / L_chol[j, j]

    return L_chol

# --- Test and Verification of Q2(b) ---
# We use the same symmetric positive definite matrix A_sym from Q2(a)
L_chol = cholesky_decomposition(A_sym)

print_matrix("Lower Triangular Cholesky Factor L_chol", L_chol)

# Verification: A = L_chol * L_chol^T
L_chol_T = custom_mat_transpose(L_chol)
L_LL_T = custom_mat_mul_mat(L_chol, L_chol_T)
print_matrix("Reconstructed Matrix L_chol * L_chol^T", L_LL_T)

error_cholesky = custom_matrix_norm_frobenius(A_sym - L_LL_T)
print(f"Cholesky Reconstruction Error ||A - L*L^T||_F = {error_cholesky:.
```

```
--- Lower Triangular Cholesky Factor L_chol ---
[  3.7417   0.0000   0.0000   0.0000  ]
[  2.9399   3.2183   0.0000   0.0000  ]
[  2.6726   0.9766   2.9839   0.0000  ]
[  3.7417   0.9322   1.7057   3.0367  ]

--- Reconstructed Matrix L_chol * L_chol^T ---
[  14.0000   11.0000   10.0000   14.0000  ]
[  11.0000   19.0000   11.0000   14.0000  ]
[  10.0000   11.0000   17.0000   16.0000  ]
[  14.0000   14.0000   16.0000   27.0000  ]

Cholesky Reconstruction Error ||A - L*L^T||_F = 0.0000e+00
```

Q2(c) QR Decomposition

Theory & Gram-Schmidt Process

Given n linearly independent vectors in m-dimensional space (arranged as the columns of an $m \times n$ matrix A with $m > n$), we can decompose A into:

$$A = QR$$

where:

- $Q \in \mathbb{R}^{m \times n}$ has **orthonormal columns** ($Q^T Q = I_n$).
- $R \in \mathbb{R}^{n \times n}$ is an **upper triangular matrix**.

We implement the **Modified Gram-Schmidt (MGS)** algorithm. Compared to the classical Gram-Schmidt (CGS) algorithm, MGS is numerically far more stable under

floating-point arithmetic because it updates the remaining un-normalized columns immediately as soon as a new orthonormal basis vector is computed.

Modified Gram-Schmidt Algorithm

Let the columns of A be $a_1, a_2, \ldots, a_n$.

1. For $j = 0, 1, \ldots, n-1$:
   - Set $v_j = a_j$.
2. For $j = 0, 1, \ldots, n-1$:
   - Compute the norm of the current column: $R_{jj} = \|v_j\|_2 = \sqrt{\sum_{k=0}^{m-1} v_{j,k}^2}$.
   - Normalize column j to get the j-th orthonormal vector: $q_j = v_j / R_{jj}$.
   - For all subsequent columns $i = j+1, \ldots, n-1$:
     - Compute the projection of column i onto q_j: $R_{ji} = q_j^T v_i$.
     - Subtract the projection immediately to orthogonalize: $v_i \leftarrow v_i - R_{ji} q_j$.

This process constructs $Q = [q_1 \; q_2 \; \ldots \; q_n]$ and the upper triangular matrix R element-by-element.

> ✍️ [HANDWRITTEN SUBMISSION NOTE]
>
> **For your handwritten PDF assignment submission (`BITSID.pdf`), you should write:**
>
> 1. **QR Mathematical Formulation:** Explain the Gram-Schmidt orthogonalization process.
> 2. **QR Code Snippet:** Paste the screenshots of `qr_decomposition` from the cell below.
> 3. **Sample Matrix & Result:** Write down a sample matrix A and its decomposed components Q and R.

Q2(c) Custom QR Decomposition Implementation

In [7]:

```python
def qr_decomposition(A):
    """
    Computes the QR decomposition of an m x n matrix A (m >= n)
    having linearly independent columns using the Modified Gram-Schmidt (
    Returns Q (m x n) with orthonormal columns, and R (n x n) upper trian
    """
    m, n = A.shape
    Q = np.zeros((m, n))
    R = np.zeros((n, n))

    # Create a working copy of A's columns
    V = A.copy().astype(np.float64)

    for j in range(n):
        # 1. Compute R[j, j] as the L2 norm of the active orthogonal vect
        norm_v = 0.0
        for k in range(m):
            norm_v += V[k, j] ** 2
```

```
        norm_v = np.sqrt(norm_v)

        R[j, j] = norm_v
        if norm_v < 1e-9:
            raise ValueError("Columns are not linearly independent! QR ca

        # 2. Compute the j-th column of Q
        Q[:, j] = V[:, j] / norm_v

        # 3. Orthogonalize the remaining columns with respect to q_j
        for i in range(j + 1, n):
            # R[j, i] = q_j^T * v_i
            dot_product = 0.0
            for k in range(m):
                dot_product += Q[k, j] * V[k, i]
            R[j, i] = dot_product

            # Subtract the projection: v_i <- v_i - R[j, i] * q_j
            V[:, i] = V[:, i] - R[j, i] * Q[:, j]

    return Q, R

# --- Test and Verification of Q2(c) ---
np.random.seed(5)
# Construct a small 5 x 3 matrix with linearly independent columns
A_test_qr = np.random.randint(-3, 4, size=(5, 3)).astype(np.float64)

Q_t, R_t = qr_decomposition(A_test_qr)
print_matrix("Input Matrix A (5 x 3)", A_test_qr)
print_matrix("Orthogonal Matrix Q (5 x 3)", Q_t)
print_matrix("Upper Triangular Matrix R (3 x 3)", R_t)

# Verify QR = A
QR_reconstructed = custom_mat_mul_mat(Q_t, R_t)
print_matrix("Reconstructed Matrix Q * R", QR_reconstructed)
error_qr = custom_matrix_norm_frobenius(A_test_qr - QR_reconstructed)
print(f"Reconstruction Error ||A - Q*R||_F = {error_qr:.4e}")

# Verify Q^T * Q = I
QT_Q = custom_mat_mul_mat(custom_mat_transpose(Q_t), Q_t)
print_matrix("Orthogonality Check Q^T * Q", QT_Q)
error_ortho = custom_matrix_norm_frobenius(QT_Q - np.eye(3))
print(f"Orthogonality Error ||Q^T*Q - I||_F = {error_ortho:.4e}")
```

```
--- Input Matrix A (5 x 3) ---
[  0.0000   3.0000   2.0000  ]
[  3.0000   3.0000  -3.0000  ]
[ -2.0000  -3.0000   1.0000  ]
[  3.0000   0.0000  -3.0000  ]
[  3.0000  -3.0000   1.0000  ]

--- Orthogonal Matrix Q (5 x 3) ---
[  0.0000   0.5083   0.6722  ]
[  0.5388   0.4099  -0.2585  ]
[ -0.3592  -0.4427  -0.1416  ]
[  0.5388  -0.0984  -0.3912  ]
[  0.5388  -0.6066   0.5553  ]

--- Upper Triangular Matrix R (3 x 3) ---
[  5.5678   1.0776  -3.0533  ]
[  0.0000   5.9024  -0.9673  ]
[  0.0000   0.0000   3.7070  ]

--- Reconstructed Matrix Q * R ---
[  0.0000   3.0000   2.0000  ]
[  3.0000   3.0000  -3.0000  ]
[ -2.0000  -3.0000   1.0000  ]
[  3.0000   0.0000  -3.0000  ]
[  3.0000  -3.0000   1.0000  ]

Reconstruction Error ||A - Q*R||_F = 4.4409e-16
--- Orthogonality Check Q^T * Q ---
[  1.0000   0.0000   0.0000  ]
[  0.0000   1.0000   0.0000  ]
[  0.0000   0.0000   1.0000  ]

Orthogonality Error ||Q^T*Q - I||_F = 1.7554e-16
```

Q2(d) Randomized 7×5 QR Decomposition & Analysis of R Diagonal

Now, we generate a random 7×5 matrix A having linearly independent columns and decompose it into Q and R. Since the matrix is random and $m > n$ ($7 > 5$), its columns are linearly independent with probability 1. We run `qr_decomposition(A)` to obtain Q and R, output their entries, and focus specifically on the diagonal elements of R (R_{ii} for $i = 0, \ldots, 4$).

> ### ✍️ [HANDWRITTEN SUBMISSION NOTE]
>
> **For your handwritten PDF assignment submission (`BITSID.pdf`), you should write:**
>
> 1. **Random Matrix A:** Copy the printed random 7×5 matrix A and its QR factors printed below.
> 2. **Diagonal Elements of R:** List the printed diagonal elements R_{ii} ($R_{11}, R_{22}, R_{33}, R_{44}, R_{55}$).
> 3. **Crucial Observations:** Write down the following detailed observations:

- **Observation 1 (Positivity):** All diagonal elements of R are strictly positive ($R_{ii} > 0$). In this specific run, they are strictly positive numbers.
- **Observation 2 (Geometric Meaning):** In the Gram-Schmidt process, the diagonal element R_{ii} represents the Euclidean (L_2) norm of the i-th column vector a_i after projecting out its components along all preceding orthonormal basis vectors $q_1, \ldots, q_{i-1}$:

$$R_{ii} = \|v_i\|_2 \quad \text{where } v_i = a_i - \sum_{k=0}^{i-1} (q_k^T a_i) q_k$$

Geometrically, R_{ii} is the **perpendicular distance** of the column vector a_i to the subspace spanned by the previous $i-1$ columns. Since the columns of A are **linearly independent**, no column a_i lies in the span of the preceding columns. Thus, the orthogonal residual vector v_i is never the zero vector, which mathematically guarantees that its norm $R_{ii} = \|v_i\|_2$ is strictly positive ($R_{ii} > 0$).

Q2(d) Execution and Verification on a Random 7×5 Matrix

In [8]:
```
np.random.seed(99)

# 1. Generate a random 7 x 5 matrix with floating-point entries in [-2.0,
A_7x5 = np.random.uniform(-2.0, 2.0, size=(7, 5))

# 2. Perform QR Decomposition
Q_7x5, R_7x5 = qr_decomposition(A_7x5)

# 3. Print inputs and outputs
print_matrix("Random Matrix A (7 x 5)", A_7x5)
print_matrix("Orthogonal Matrix Q (7 x 5)", Q_7x5)
print_matrix("Upper Triangular Matrix R (5 x 5)", R_7x5)

# 4. Focus on diagonal elements of R
print("=== DIAGONAL ELEMENTS OF R ===")
diag_R = []
for i in range(5):
    val = R_7x5[i, i]
    diag_R.append(val)
    print(f"R_{i+1}{i+1} = {val:.6f}")
print()

# 5. Verification
QR_7x5_rec = custom_mat_mul_mat(Q_7x5, R_7x5)
error_rec_7x5 = custom_matrix_norm_frobenius(A_7x5 - QR_7x5_rec)
print(f"Reconstruction Error ||A - Q*R||_F = {error_rec_7x5:.4e}")

QT_Q_7x5 = custom_mat_mul_mat(custom_mat_transpose(Q_7x5), Q_7x5)
error_ortho_7x5 = custom_matrix_norm_frobenius(QT_Q_7x5 - np.eye(5))
print(f"Orthogonality Error ||Q^T*Q - I||_F = {error_ortho_7x5:.4e}")
print()
```

```
# 6. Print written observation explicitly
print("=== FORMAL OBSERVATIONS ===")
print("1. Positivity of R_ii:")
print("   Every diagonal element of R is strictly positive (R_ii > 0). In
print(f"   {', '.join([f'{x:.6f}' for x in diag_R])}")
print("2. Geometric Interpretation:")
print("   The diagonal element R_ii = ||v_i||_2 is the Euclidean norm of
print("   which is the orthogonal component of column a_i that remains af
print("   span of the first i-1 columns. Geometrically, it represents the
print("   from the vector a_i to the subspace spanned by the preceding co
print("   of A are linearly independent, a_i can never lie in the span of
print("   so v_i != 0, ensuring its norm R_ii is strictly positive.")
```

```
--- Random Matrix A (7 x 5) ---
[  0.6891  -0.0477   1.3020  -1.8742   1.2322  ]
[  0.2625  -0.8095  -1.8132   1.9625  -1.9727  ]
[  1.0792   0.9871  -0.4902  -0.0234   1.7158  ]
[ -0.4182   1.8958   0.0977  -1.6255   1.2532  ]
[ -1.1533   0.2174  -0.8309   1.2646   1.3122  ]
[ -1.1137   0.5793  -1.6193  -0.3533  -1.6125  ]
[ -1.4240  -1.1512  -0.0934  -1.6895  -1.0598  ]

--- Orthogonal Matrix Q (7 x 5) ---
[  0.2707  -0.0495   0.3701  -0.4288   0.2988  ]
[  0.1031  -0.3214  -0.7187  -0.1156  -0.0172  ]
[  0.4239   0.3283  -0.3380  -0.2999   0.5676  ]
[ -0.1643   0.7438   0.1253  -0.1741  -0.1118  ]
[ -0.4530   0.1355  -0.1216   0.5131   0.6480  ]
[ -0.4375   0.2721  -0.4135  -0.3677  -0.2684  ]
[ -0.5593  -0.3754   0.1755  -0.5330   0.2895  ]

--- Upper Triangular Matrix R (5 x 5) ---
[  2.5458   0.3026   1.0786   0.4788   1.3554  ]
[  0.0000   2.6158  -0.0882  -1.0451   2.2053  ]
[  0.0000   0.0000   2.7171  -2.6040   1.7720  ]
[  0.0000   0.0000   0.0000   2.5462   0.7979  ]
[  0.0000   0.0000   0.0000   0.0000   2.2120  ]

=== DIAGONAL ELEMENTS OF R ===
R_11 = 2.545822
R_22 = 2.615843
R_33 = 2.717125
R_44 = 2.546212
R_55 = 2.211966

Reconstruction Error ||A - Q*R||_F = 6.8439e-16
Orthogonality Error ||Q^T*Q - I||_F = 5.5233e-16

=== FORMAL OBSERVATIONS ===
1. Positivity of R_ii:
   Every diagonal element of R is strictly positive (R_ii > 0). In this ru
n, the values are:
   2.545822, 2.615843, 2.717125, 2.546212, 2.211966
2. Geometric Interpretation:
   The diagonal element R_ii = ||v_i||_2 is the Euclidean norm of the resi
dual vector v_i,
   which is the orthogonal component of column a_i that remains after proj
ecting it onto the
   span of the first i-1 columns. Geometrically, it represents the perpend
icular distance
   from the vector a_i to the subspace spanned by the preceding columns. B
ecause the columns
   of A are linearly independent, a_i can never lie in the span of the pre
ceding columns,
   so v_i != 0, ensuring its norm R_ii is strictly positive.
```